

NoSQL Database for Credit-Card Fraud Detection

Detailed Implementation Report & Advanced Engineering Guide — MongoDB

Build-it report: runnable code for generation, loading, all five operations, schema extension & benchmarking | **+ Next level:** schema validation, geospatial signals, change streams, materialized views, sharding, dashboard, Neo4j comparison | **Stack:** MongoDB 7.x · Python 3.11 · pymongo | **Date:** 30 May 2026

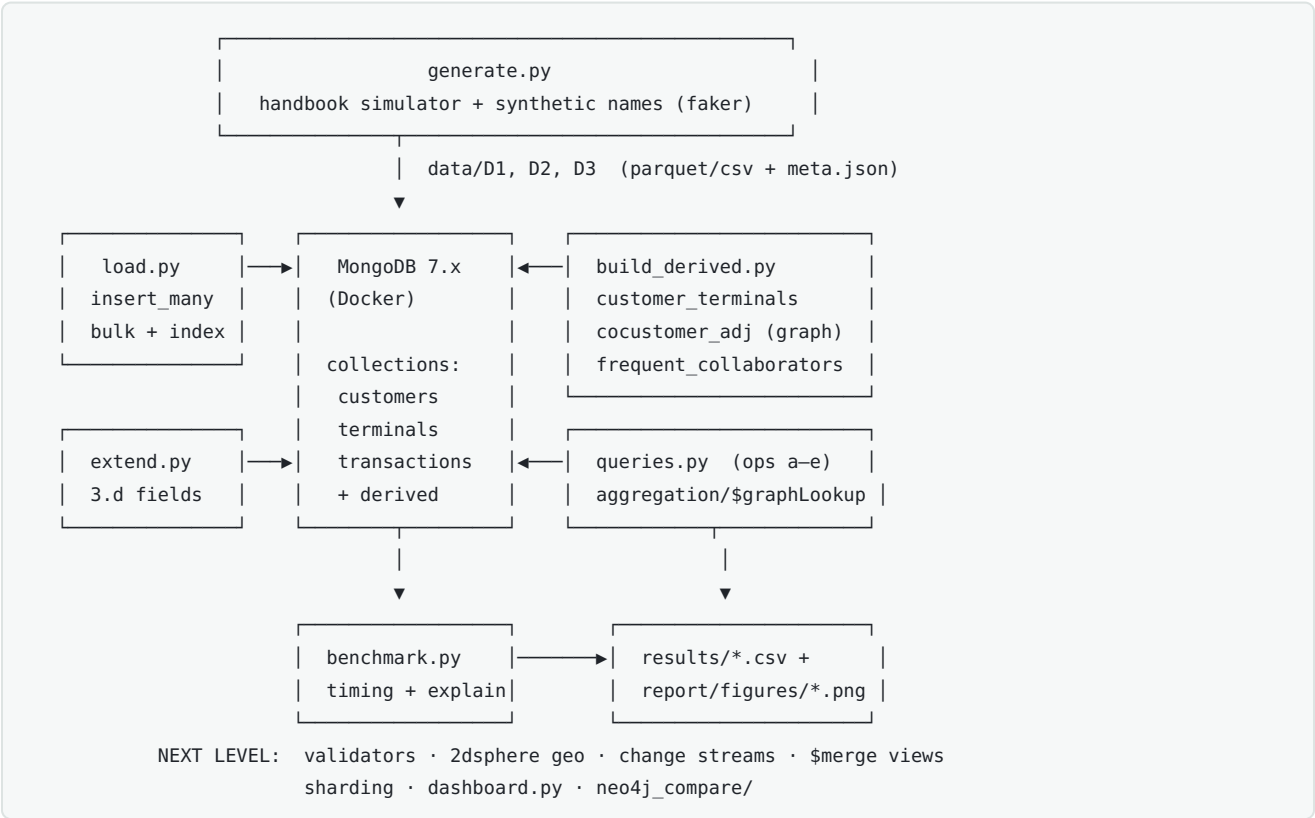
Purpose: an implementation-ready technical report that (1) shows exactly how to build the project end-to-end with runnable code, and (2) takes it beyond the baseline with advanced engineering that earns extra credit. **Target system:** MongoDB 7.x · **Language:** Python 3.11 (pymongo) · **Team:** ≤ 2 · **Valid until:** April 2027

1. Objectives and grading-aligned scope

The assignment asks for a NoSQL database populated from the handbook transaction simulator, a conceptual and logical model, five non-trivial operations, a post-load schema extension, and a performance study over three datasets (≥50/100/200 MB). This report delivers a concrete construction path for all of that, then adds an **"engineering at the next level"** layer — schema validation, geospatial fraud signals, real-time change-stream alerting, materialized incremental views, a sharding strategy, a results dashboard, and a head-to-head comparison with Neo4j for the graph query.

Everything is mapped back to the deliverable list in Section 13 so nothing required is dropped while the extras are clearly additive.

2. System architecture



The pipeline is intentionally staged so each phase is independently runnable and benchmarkable, which is exactly what the performance section needs.

3. Environment and reproducibility

`docker-compose.yml` — a pinned database so results reproduce on any machine:

```
services:
  mongo:
    image: mongo:7.0
    container_name: fraud_mongo
    ports: ["27017:27017"]
    command: ["--wiredTigerCacheSizeGB", "2"] # bound cache for fair benchmarks
    volumes: ["./.mongodata:/data/db"]
```

`requirements.txt`:

```
pymongo==4.8.0
pandas==2.2.2
numpy==1.26.4
matplotlib==3.9.0
faker==25.0.0
tqdm==4.66.4
```

`Makefile` (one-command reproduction):

```
up:      ; docker compose up -d
gen:     ; python src/generate.py --all
load:    ; python src/load.py --dataset $(D)
derive:  ; python src/build_derived.py --dataset $(D)
extend:  ; python src/extend.py --dataset $(D)
bench:   ; python src/benchmark.py --all
all: up gen load derive extend bench
```

Fix the random seed everywhere (`numpy`, `faker`, simulator) so the three datasets and the benchmarks are deterministic — this is what makes the performance discussion defensible.

4. Dataset generation (Activity 1)

`src/generate.py` reuses the handbook functions and patches in the missing `name` field required by query (a).

```

import numpy as np, pandas as pd, os, json
from faker import Faker
# vendored from the handbook (Chapter 3, Section 2):
from simulator.handbook import (generate_customer_profiles_table,
                                generate_terminal_profiles_table,
                                get_list_terminals_within_radius,
                                generate_transactions_table, add_frauds)

def generate_dataset(n_customers, n_terminals, nb_days, start_date, r, seed=42):
    fake = Faker(); Faker.seed(seed); np.random.seed(seed)
    customers = generate_customer_profiles_table(n_customers, random_state=seed)
    terminals = generate_terminal_profiles_table(n_terminals, random_state=seed)
    # geographic reachability -> available_terminals
    x_y_terminals = terminals[['x_terminal_id', 'y_terminal_id']].values.astype(float)
    customers['available_terminals'] = customers.apply(
        lambda c: get_list_terminals_within_radius(c, x_y_terminals, r), axis=1)
    customers['nb_terminals'] = customers['available_terminals'].apply(len)
    customers['name'] = [fake.name() for _ in range(len(customers))] # <- patch
    # transactions
    tx = customers.groupby('CUSTOMER_ID').apply(
        lambda c: generate_transactions_table(c.iloc[0], start_date, nb_days)
    ).reset_index(drop=True)
    tx = add_frauds(customers, terminals, tx) # TX_FRAUD labels
    return customers, terminals, tx

def write_dataset(name, customers, terminals, tx, params):
    d = f"data/{name}"; os.makedirs(d, exist_ok=True)
    customers.to_parquet(f"{d}/customers.parquet")
    terminals.to_parquet(f"{d}/terminals.parquet")
    tx.to_parquet(f"{d}/transactions.parquet")
    size_mb = os.path.getsize(f"{d}/transactions.parquet")/1e6
    json.dump(**params, {"tx_rows": len(tx), "size_mb_tx": round(size_mb,1)},
              open(f"{d}/meta.json", "w"), indent=2)
    return size_mb

```

Sizing by calibration. Total transactions $\approx$ `n_customers × mean_nb_tx_per_day × nb_days`. With 1 000 customers and the handbook default rate (~2/day) you get ~2 000 tx/day, so:

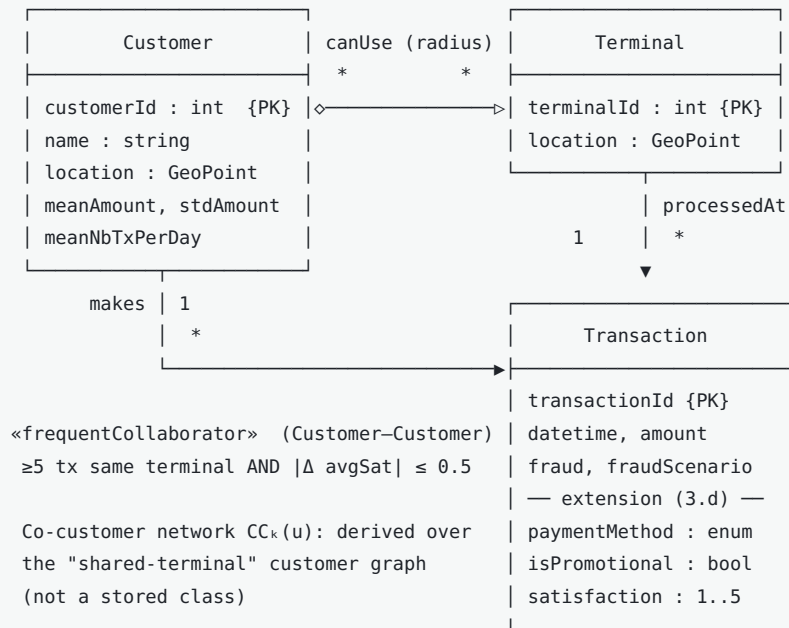
Dataset	Target	Customers	Terminals	<code>nb_days</code> (start)	Lever if short
D1	≥50 MB	1 000	200	~250	raise <code>nb_days</code> or rate
D2	≥100 MB	1 000	200	~500	raise <code>nb_days</code>
D3	≥200 MB	1 000	200	~1000	raise <code>nb_days</code> /customers

Generate → read `meta.json` → if `size_mb_tx < target` bump `nb_days` and regenerate. Record the **measured CSV/Parquet size** (the natural "dataset size") in the report; that is the figure graded against the 50/100/200 MB requirement.

Decisions to document: radius `r` (controls terminal-sharing density → directly affects how hard queries a and c are), `start_date`, seed, and the name-generation strategy.

5. Conceptual model (Activity 2)

Three core classes; the *frequent-collaborator* edge and the transaction extensions are added in step 3.d. Produce the diagram twice — before and after — to show the model's evolution.



Constraints to state explicitly: `satisfaction ∈ [1,5]`, `amount ≥ 0`, `paymentMethod ∈ {credit_card, mobile_payment, paypal}`, `fraud ∈ {0,1}`; `makes` and `processedAt` are mandatory single-valued; `frequentCollaborator` is symmetric and derived.

6. Logical model and schema validation (Activity 3 + next level)

6.1 Collections

Three base collections (reference, don't embed — transactions are unbounded) plus derived collections used purely for speed.

```

# customers
{ "_id": 0, "name": "Dana Lopez",
  "location": {"type":"Point","coordinates":[54.8, 51.4]}, # GeoJSON (next level)
  "meanAmount": 62.3, "stdAmount": 31.1, "meanNbTxPerDay": 2.18,
  "availableTerminals": [12,88,140,199], "nbTerminals": 4 }

# terminals
{ "_id": 12, "location": {"type":"Point","coordinates":[50.1,48.9]} }

# transactions
{ "_id": 991234, "customerId": 0, "terminalId": 12,
  "datetime": ISODate("2025-06-03T14:22:10Z"), "amount": 71.4,
  "fraud": 0, "fraudScenario": 0,
  "paymentMethod":"paypal", "isPromotional": false, "satisfaction": 4, # 3.d
  "year":2025, "quarter":2, "dayOfWeek":3, "isOutlier": false } # computed
  
```

6.2 Next level — enforce a schema with `$jsonSchema` validators

Document stores are schemaless, but a graded project looks far stronger if the database **rejects malformed data**. Attach a validator at collection creation:

```
db.create_collection("transactions", validator={"$jsonSchema": {
  "bsonType": "object",
  "required": ["_id", "customerId", "terminalId", "datetime", "amount", "fraud"],
  "properties": {
    "amount": { "bsonType": ["double", "int"], "minimum": 0 },
    "fraud": { "enum": [0, 1] },
    "satisfaction": { "bsonType": ["int", "null"], "minimum": 1, "maximum": 5 },
    "paymentMethod": { "enum": ["credit_card", "mobile_payment", "paypal", None] }
  }
}, validationLevel="moderate")
```

6.3 Derived collections and the patterns behind them

Collection	Pattern	Why
<code>customer_terminals { _id, terminals[], txCount }</code>	Computed	feeds queries (a) and (c) without rescanning transactions
<code>cocustomer_adj { _id, neighbors[] }</code>	Materialized graph	enables <code>\$graphLookup</code> for CC _k (query c)
<code>terminal_prev_median { _id, median }</code>	Computed / Bucket	precomputes query (b)'s baseline
<code>frequent_collaborators { a, b, terminal, avgSatA, avgSatB }</code>	Materialized relationship	makes 3.d.ii directly queryable
<code>transactions.year/quarter/dayOfWeek/isOutlier</code>	Computed	removes per-query date math (b, e)

The report must compare **naive vs derived** runtimes — that contrast is the heart of the performance argument.

6.4 Index plan

```
db.transactions.create_index([("customerId", 1)])
db.transactions.create_index([("terminalId", 1), ("datetime", 1)])
db.transactions.create_index([("quarter", 1)])
db.transactions.create_index([("terminalId", 1), ("customerId", 1)])
db.customers.create_index([("availableTerminals", 1)]) # multikey
db.customers.create_index([("location", "2dsphere")]) # geo (next level)
db.terminals.create_index([("location", "2dsphere")])
db.cocustomer_adj.create_index([("_id", 1)])
db.frequent_collaborators.create_index([("a", 1)]); db.frequent_collaborators.create_index([("b", 1)])
```

Build indexes **after** bulk load and confirm usage with `explain("executionStats")`.

7. Loading pipeline (Activity 4)

`src/load.py` — batched bulk insert, indexes last:

```

import pandas as pd, pymongo, time
from pymongo import MongoClient, InsertOne

def load(dataset, uri="mongodb://localhost:27017", db_name=None):
    db = MongoClient(uri)[db_name or f"fraud_{dataset}"]
    t0 = time.perf_counter()
    # terminals + customers (small) -----
    term = pd.read_parquet(f"data/{dataset}/terminals.parquet")
    db.terminals.insert_many([
        {"_id": int(r.TERMINAL_ID),
         "location": {"type": "Point", "coordinates": [float(r.x_terminal_id), float(r.y_terminal_id)]}}
        for r in term.itertuples()], ordered=False)
    cust = pd.read_parquet(f"data/{dataset}/customers.parquet")
    db.customers.insert_many([
        {"_id": int(r.CUSTOMER_ID), "name": r.name,
         "location": {"type": "Point", "coordinates": [float(r.x_customer_id), float(r.y_customer_id)]},
         "meanAmount": float(r.mean_amount), "stdAmount": float(r.std_amount),
         "meanNbTxPerDay": float(r.mean_nb_tx_per_day),
         "availableTerminals": list(map(int, r.available_terminals)),
         "nbTerminals": int(r.nb_terminals)} for r in cust.itertuples()], ordered=False)
    # transactions (large) - stream in batches -----
    tx = pd.read_parquet(f"data/{dataset}/transactions.parquet")
    tx["datetime"] = pd.to_datetime(tx["TX_DATETIME"])
    ops, BATCH = [], 10_000
    for r in tx.itertuples():
        ops.append(InsertOne({
            "_id": int(r.TRANSACTION_ID), "customerId": int(r.CUSTOMER_ID),
            "terminalId": int(r.TERMINAL_ID), "datetime": r.datetime.to_pydatetime(),
            "amount": float(r.TX_AMOUNT), "fraud": int(r.TX_FRAUD),
            "fraudScenario": int(r.TX_FRAUD_SCENARIO),
            "year": r.datetime.year, "quarter": (r.datetime.month-1)//3+1,
            "dayOfWeek": r.datetime.isoweekday(), "isOutlier": False}))
    if len(ops) == BATCH:
        db.transactions.bulk_write(ops, ordered=False); ops = []
    if ops: db.transactions.bulk_write(ops, ordered=False)
    build_indexes(db) # AFTER load
    return time.perf_counter() - t0 # load time -> benchmark

```

Record load time per dataset; it is explicitly part of Activity 5.

8. The five operations (Activity 3 a–e) — full implementations

8.a Similar customers by shared terminals

```
def build_customer_terminals(db):
    # Computed Pattern, once
    db.customer_terminals.drop()
    db.transactions.aggregate([
        {"$group":{"_id":"$customerId",
                    "terminals":{"$addToSet":"$terminalId"},
                    "txCount":{"$sum":1}}},
        {"$merge":{"into":"customer_terminals"}}], allowDiskUse=True)

def similar_customers(db, M):
    m = db.customer_terminals.find_one({"_id": M})
    if not m: return []
    return list(db.customer_terminals.aggregate([
        {"$match":{"_id":{"$ne":M},
                    "txCount":{"$gte":m["txCount"]-2,"$lte":m["txCount"]+2}}}, # cheap prune first
        {"$project":{"txCount":1,
                      "shared":{"$size":{"$setIntersection":["$terminals", m["terminals"]]}},
                      "$match":{"shared":{"$gte":4}}},
        {"$lookup":{"from":"customers","localField":"_id",
                    "foreignField":"_id","as":"c"}},
        {"$project":{"nameN":{"$first":"$c.name"},"sharedTerminals":"$shared",
                    "txCountN":"$txCount"}}], allowDiskUse=True))
```

Pruning by the ± 2 transaction-count window *before* the set-intersection keeps this far below the naive $O(n^2)$. For "all M" run the function across customers, or batch with a `$lookup` self-join; the report compares both.

8.b Quarterly per-terminal outliers

```
def mark_outliers(db, cur=(2025,2), prev=(2025,1)):
    # 1) previous-quarter median per terminal (MongoDB 7.0 $median; fallback below)
    db.terminal_prev_median.drop()
    db.transactions.aggregate([
        {"$match":{"year":prev[0],"quarter":prev[1]}},
        {"$group":{"_id":"$terminalId",
                    "median":{"$median":{"input":"$amount","method":"approximate"}}}},
        {"$merge":{"into":"terminal_prev_median"}}], allowDiskUse=True)
    # 2) flag current-quarter tx > 1.3 * prev median -> isOutlier
    db.transactions.aggregate([
        {"$match":{"year":cur[0],"quarter":cur[1]}},
        {"$lookup":{"from":"terminal_prev_median","localField":"terminalId",
                    "foreignField":"_id","as":"m"}},
        {"$set":{"medPrev":{"$first":"$m.median"}}},
        {"$match":{"$expr":{"$gt":["$amount",{"$multiply":["$medPrev",1.3]}]}},
        {"$set":{"isOutlier":True}},
        {"$merge":{"into":"transactions","whenMatched":"merge"}}], allowDiskUse=True)
```

Fallback median for MongoDB < 7.0: `$push` amounts, `$sortArray`, take the middle element. Index `{terminalId:1, datetime:1}` (or `{quarter:1}`) is what makes the quarter scan cheap.

8.c Co-customer network $CC_k (\geq CC_3)$ via `$graphLookup`

```
def build_cocustomer_adj(db):
    # materialized graph, once
    db.cocustomer_adj.drop()
    db.transactions.aggregate([
        {"$group":{"_id":"$terminalId","custs":{"$addToSet":{"customerId"}}},
        {"$project":{"custs":1}},
        {"$unwind":"$custs"},
        {"$project":{"cust":"$custs",
            "others":{"$setDifference":["$custs",["$custs"]]}},
        {"$unwind":"$others"},
        {"$group":{"_id":"$cust","neighbors":{"$addToSet":{"$others"}}},
        {"$merge":{"into":"cocustomer_adj"}}, allowDiskUse=True)

def co_customer_network(db, u, k=3):
    # maxDepth = k-2 -> CC3 (2 hops) => maxDepth 1
    res = list(db.cocustomer_adj.aggregate([
        {"$match":{"_id":u}},
        {"$graphLookup":{"from":"cocustomer_adj","startWith":"$neighbors",
            "connectFromField":"neighbors","connectToField":"_id",
            "as":"net","maxDepth":k-2,"depthField":"deg"}},
        {"$project":{"cc":{"$setDifference":
            [{"$map":{"input":"$net","in":"$$this._id"}}, [u]]}}}))
    return res[0]["cc"] if res else []
```

The path semantics ("distinct customers connected through shared terminals") map directly onto `$graphLookup` over the materialized adjacency. **Watch the edge count** — popular terminals create near-complete subgraphs; mitigations (cap customers/terminal, bucket) are discussed in the performance section, and the Neo4j comparison in Section 12 shows where a native graph engine pulls ahead here.

8.d Schema extension — run *after* loading

```
def extend_transactions(db):
    # 3.d.i, server-side, fast
    db.transactions.update_many({}, [{"$set":{"
        "paymentMethod":{"$arrayElemAt":["credit_card","mobile_payment","paypal"],
        {"$toInt":{"$floor":{"$multiply":[{"$rand":{}},3]}}}},
        "isPromotional":{"$lt":[{"$rand":{}},0.3]},
        "satisfaction":{"$toInt":{"$add":[1,{"$floor":{"$multiply":[{"$rand":{}},5]}}]}}
    }]])

def build_frequent_collaborators(db):
    # 3.d.ii, queryable relationship
    db.frequent_collaborators.drop()
    per_term = db.transactions.aggregate([
        {"$group":{"_id":{"c":"$customerId","t":"$terminalId"},
            "n":{"$sum":1},"avgSat":{"$avg":"$satisfaction"}}},
        {"$match":{"n":{"$gte":5}}},
        {"$group":{"_id":"$_id.t",
            "members":{"$push":{"c":"$_id.c","avgSat":"$avgSat"}}}},
        allowDiskUse=True)
    edges = []
    for t in per_term:
        # pair within terminal
        m = t["members"]
        for i in range(len(m)):
            for j in range(i+1, len(m)):
                if abs(m[i]["avgSat"]-m[j]["avgSat"]) <= 0.5:
                    edges.append({"a":m[i]["c"],"b":m[j]["c"],"terminal":t["_id"],
                        "avgSatA":m[i]["avgSat"],"avgSatB":m[j]["avgSat"]})
    if edges: db.frequent_collaborators.insert_many(edges)
```


The extension is genuinely a *post-load* operation, the field updates are done server-side with `$rand` (one pass, no Python loop), and the collaborator relationship is materialized + indexed so it is directly queryable as the brief requires.

8.e Day-of-week summary table

```
def weekday_summary(db):
    return list(db.transactions.aggregate([
        {"$group": {"_id": "$dayOfWeek", "total": {"$sum": 1},
            "outliers": {"$sum": {"$cond": [{"$isOutlier", 1, 0}]}}}},
        {"$project": {"total": 1, "outliers": 1,
            "outlierPct": {"$round": [{"$multiply": [
                {"$divide": ["$outliers", "$total"]}, 100], 2]}}}},
        {"$sort": {"_id": 1}}])]) # Mon..Sun
```

Depends on the `isOutlier` flag set in 8.b; renders as the required 7-row summary table.

9. Benchmarking framework (Activity 5)

`src/benchmark.py` times every operation (including load and the 3.d update) across D1/D2/D3, repeats for stable numbers, and emits CSVs + figures.

```
import time, statistics, csv, matplotlib.pyplot as plt

def timed(fn, *a, repeats=3, **k):
    ts = []
    for _ in range(repeats):
        t0 = time.perf_counter(); fn(*a, **k); ts.append(time.perf_counter()-t0)
    return {"mean": statistics.mean(ts), "best": min(ts)}

OPS = {"load":..., "derive":..., "q_a":..., "q_b":..., "q_c":...,
    "q_d_extend":..., "q_e":...}

def run_all(datasets=("D1", "D2", "D3")):
    rows = []
    for ds in datasets:
        db = connect(ds)
        for name, fn in OPS.items():
            rows.append({"dataset": ds, "op": name, **timed(fn, db)})
    csv.DictWriter(open("results/timings.csv", "w"),
        fieldnames=rows[0].keys()).writerows(rows)
    plot(rows)
```

Required figures: load time vs size; grouped bars of each query across datasets; **naive-vs-optimized** for the heavy queries (a, c); index-on vs index-off for (b); plus the (e) summary table. Capture `explain("executionStats")` (docs examined, index used, disk vs memory) to back the discussion with evidence rather than assertion.

10. Taking it to the next level — advanced engineering

These are additive: each is small to implement but visibly raises the project's ceiling. Implement the ones your time allows and present them as a clearly-labelled "extensions" chapter.

10.1 Geospatial fraud signal (`2dsphere`)

Storing customer and terminal locations as GeoJSON and adding a `2dsphere` index unlocks a real fraud heuristic absent from the base spec: **transactions occurring far from a customer's home location**. A `$geoNear` /

`$geoWithin` query flags "impossible travel" or out-of-radius spends — a meaningful analytic that also demonstrates a second index type.

```
db.terminals.aggregate([{"$geoNear":{
  "near": customer_home_point, "distanceField":"dist",
  "maxDistance": 30, "spherical": True}}])
```

10.2 Real-time outlier alerting with change streams

A `watch()` cursor turns the batch outlier rule (8.b) into a **streaming detector**: every inserted transaction is checked against the terminal's stored median and flagged live. This shows MongoDB as an operational FDS, not just an analytics store.

```
with db.transactions.watch([{"$match":{"operationType":"insert"}}]) as stream:
    for change in stream:
        tx = change["fullDocument"]
        med = db.terminal_prev_median.find_one({"_id":tx["terminalId"]})
        if med and tx["amount"] > 1.3*med["median"]:
            db.alerts.insert_one({"txId":tx["_id"],"reason":"outlier","ts":tx["datetime"]})
```

10.3 Incremental materialized views with `$merge`

Rather than rebuilding `customer_terminals/terminal_prev_median` from scratch, schedule an **incremental `$merge`** over only the latest window (e.g. nightly). This is the production-grade version of the Computed Pattern and a strong talking point for the performance chapter.

10.4 Sharding strategy for horizontal scale

Discuss (and optionally demo on a small sharded set) shard-key choice: a **hashed `customerId`** spreads writes and keeps per-customer queries (a, c, d) targeted, whereas `terminalId` favours queries (b, e) but risks hot terminals. Cover chunk balancing and the trade-off — this directly addresses how the design scales past 200 MB.

10.5 Time-series collections (optional)

MongoDB time-series collections compress transaction streams keyed by `datetime/terminalId` and accelerate the quarterly aggregations (b, e). Note the update constraints versus a regular collection and present it as an alternative physical model with its own benchmark.

10.6 Results dashboard

A small `dashboard.py` (Streamlit or a static HTML built from the benchmark CSVs) that renders the timing charts, the weekday table, and a co-customer network preview. A visual artifact makes the oral/defence far stronger and shows the data is genuinely queryable end-to-end.

10.7 Cross-engine comparison: MongoDB vs Neo4j for query (c)

Implement CC₃ in Cypher (`MATCH (u)-[:SHARES*1..2]-(y)`) on the same data and benchmark it against `$graphLookup`. Reporting *where the document model is strong (b, d, e) and where a native graph wins (c)* is exactly the kind of critical analysis that earns top marks — and it justifies, with evidence, the MongoDB choice for the overall workload.

10.8 Engineering hygiene

Covered queries (project only indexed fields), `allowDiskUse` on large pipelines, write/read concern notes, authentication enabled in `docker-compose`, seeded determinism, a `tests/` smoke suite asserting each query returns the expected shape, and a CI job (`make all`) that regenerates everything from zero.

11. Repository structure

```
fraud-nosql-mongodb/
├─ README.md  Makefile  docker-compose.yml  requirements.txt
├─ src/
│   └─ simulator/handbook.py          # vendored simulator
│   └─ generate.py  load.py  build_derived.py  extend.py
│   └─ queries.py  benchmark.py  dashboard.py
│   └─ validators.py                  # $jsonSchema definitions
├─ neo4j_compare/  cypher_queries.cypher  compare.py      # next level
├─ data/{D1,D2,D3}/                  # datasets + meta.json
├─ results/timings.csv
├─ report/figures/*.png  technical_documentation.pdf
└─ tests/test_queries.py
```

12. Performance discussion — what to write

For each operation: report mean/best across D1/D2/D3, state the scaling behaviour (linear with tx count for b/e; near-linear with pruning for a; super-linear/edge-bound for c), and back claims with `explain` stats. Show the **naive-vs-optimized delta** for (a) and (c), the **index-on/off delta** for (b), and the **MongoDB-vs-Neo4j delta** for (c). Conclude with which schema-design patterns (Computed, Bucket, Materialized graph, Extended Reference) paid off and why — that synthesis is the graded outcome of Activity 5.

13. Deliverable checklist (maps to "What to deliver")

Technical PDF must contain: UML class diagram with motivations/constraints incl. the 3.d change ✓ (§5); logical data model with motivations ✓ (§6); description of the loading script ✓ (§7); description of the operation scripts ✓ (§8); performance discussion over the three datasets incl. loading, pattern optimizations, and **comparison graphics** ✓ (§9, §12). **Plus:** all scripts ✓ (§4–§9, §10); reproduction instructions ✓ (§3, README/Makefile/Docker/seeds); everything zipped ✓. **Above baseline (extra credit):** §6.2 validators, §10 geospatial, change streams, incremental views, sharding, time-series, dashboard, and the Neo4j comparison.

14. Risks and mitigations

Risk	Mitigation
Query (a) $O(n^2)$ blow-up	<code>customer_terminals</code> + ± 2 count-window prune before set-intersection
<code>cocustomer_adj</code> edge explosion (c)	cap customers/terminal, bucket, <code>allowDiskUse</code> ; show Neo4j contrast
<code>\$median</code> needs MongoDB ≥ 7.0	pin 7.x in Docker; manual median fallback
Aggregation 100 MB stage limit	<code>allowDiskUse=True</code> , lean projections, indexed <code>\$match</code> first
Dataset not exactly on size target	calibration loop; record measured size $\geq$ target in <code>meta.json</code>
Missing names in simulator	add synthetic <code>name</code> at generation (§4)
Non-reproducible benchmarks	fixed seeds, Dockerized DB, repeats, <code>explain</code> evidence
Scope creep from extensions	extensions are clearly optional/additive; baseline ships first

15. References

1. Le Borgne, Siblini, Lebichot, Bontempi — *Reproducible Machine Learning for Credit Card Fraud Detection — Practical Handbook*, Université Libre de Bruxelles. Simulator: Chapter 3, Section 2.
2. MongoDB Manual — Aggregation Pipeline, `$graphLookup`, `$merge`, schema validation, `2dsphere` indexes, change streams, time-series collections, sharding.
3. MongoDB — *Building with Patterns* (Computed, Bucket, Extended Reference, Subset).